

# KGUPC 2025 에디토리얼

by

Official Solutions

Solutio in Kyonggi Univ. KGUPC 2025 출제진

 정보통신기획평가원

 경기대학교 | 소프트웨어중심대학사업단



| 문제       |                       | 예상 난이도           | 출제자 |
|----------|-----------------------|------------------|-----|
| <b>A</b> | 경기대학교 AI 컴퓨터공학부 & 솔루션 | <b>Bronze5</b>   | 전민주 |
| <b>B</b> | 주차장 정산 시스템            | <b>Silver5</b>   | 박준형 |
| <b>C</b> | 정렬의 사회성               | <b>Silver4</b>   | 장영후 |
| <b>D</b> | 컨벤션 대탈출               | <b>Silver2</b>   | 전민주 |
| <b>E</b> | Even하게 익은 스테이크        | <b>Silver1</b>   | 노태윤 |
| <b>F</b> | 이기적인 별자리              | <b>Gold2</b>     | 노태윤 |
| <b>G</b> | 학교 가기 싫어!             | <b>Platinum2</b> | 노태윤 |



## A. 경기대학교 AI 컴퓨터공학부 & 솔루션

출제자: 전민주 (@mingmingmon)

태그: #math, #arithmetic

예상 난이도: **Bronze5**

- ✓ 제출 32번, 정답 25명 (정답률 78.1%)
- ✓ 처음 푼 사람: 김0건, 김0민, 김0원, 박0민, 안0현 1분



- ✓  $n$ 이 19,471,108 이하의 자연수이기 때문에  
나머지 연산자 모듈러 (%)를 사용하여 짝수인지, 홀수인지 판단하면 됩니다.
- ✓ 2로 나눈 나머지가 0이라면 짝수,  
나머지가 1이라면 홀수이므로  
각각 "KGU SOLUTIO" 또는 "KGU AI CSE"를 출력하면 됩니다.
- ✓ if문을 사용하셔도 되고, 한줄로 표현하고 싶다면  
삼항 연산자로 표현하셔도 좋습니다.



## B. 주차장 정산 시스템

출제자: 박준형 (@refill447)

태그: #set, #hash\_set, #tree\_set

예상 난이도: **Silver5**

- ✓ 제출 33번, 정답 18명 (정답률 54.5%)
- ✓ 처음 푼 사람: 김0건 9분



- ✓ 핵심은 각 차량의 입차/출차 시간의 차이를 계산하는 것입니다.
  1. 차량별로 입차 시간을 저장한다.
  2. 출차 기록을 만나면 주차 시간을 계산한다.
  3. 계산된 주차 시간을 저장한다.
  4. 모든 기록 처리가 끝나면 각 차량의 요금을 계산한다.
  5. 차량 번호 기준으로 정렬 후 출력한다.

## B. 주차장 정산 시스템



### ✓ 구현 (파이썬 코드)

```
1  N = int(input())
2  in_time = {}
3  parking_time = {}
4
5  for _ in range(N):
6      t, state, car = input().split()
7      t = int(t)
8      if state == "IN":
9          in_time[car] = t
10     else: # OUT
11         duration = t - in_time[car]
12         parking_time[car] = duration
13         del in_time[car]
14
15     for car in sorted(parking_time.keys()):
16         fee = 1000 + 5 * parking_time[car]
17         print(car, fee)
```

### ✓ 시간 복잡도

해시맵 사용 시,

기록 처리:  $O(1) \cdot O(N) = O(N)$

정렬:  $O(M \log M)$  ( $M = \text{차량 수} = N/2$ )

결과:  $O(N \log N)$

트리맵 사용 시,

기록 처리:  $O(\log N) \cdot O(N) = O(N \log N)$

결과:  $O(N \log N)$



### ✓ 별해

$N \leq 1\,000$  이므로,

IN에 대해 모든 {car\_number,in\_time} 쌍을 배열에 담고,

OUT에 대해 브루트포스로 차량 번호를 찾아

in\_time과 비교하여 계산할 수 있습니다.

따라서 비용을 계산하는 데  $O(N^2)$ 의 비용이 듭니다.

마찬가지로  $O(N^2)$  정렬을 사용할 수 있습니다.





## C. 정렬의 사회성

출제자: 장영후 (@yhjang04)

태그: #string, #sorting, #implementation

예상 난이도: **Silver4**

- ✓ 제출 49번, 정답 10명 (정답률 20.4%)
- ✓ 처음 푼 사람: 김0건 19분



- ✓ 가장 중요한 점은 두 단어의 대소비교 함수를 구현하는 것입니다.  
문자별 인덱스를 기록하여, 이 인덱스로 비교하는 것이 정해입니다.
- ✓ 다만 문자열의 길이 비교까지도 통상적인 정렬 방식과 다름을 유의하여야 합니다.
- ✓ 직접 정렬을 구현해서 사용하거나, 혹은 각 언어별로 지원하는 정렬 함수를 사용하여도 됩니다.  
단어의 개수가 1,000개 이하이므로 대부분의 경우  $O(n^2)$  으로 정렬을 구현하더라도 통과할 수 있습니다.



## D. 컨벤션 대탈출

출제자: 전민주 (@mingmingmon)

태그: #bfs, #grid\_graph

예상 난이도: **Silver2**

- ✓ 제출 39번, 정답 13명 (정답률 33.3%)
- ✓ 처음 푼 사람: 김0건 29분

#### D. 컨벤션 대탈출



- ✓ 주어진 컨벤션 센터의 좌석 정보에 따르면 'O'는 이동 가능한 영역, 'X'는 이동 불가능한 영역입니다.
- ✓ 문제에서 원하는 것은 가장 왼쪽 아래에서부터 가장 오른쪽 위까지 이동하는 최단 거리입니다.
- ✓ 다양한 최단경로 알고리즘 (BFS, 플로이드 워셜, 데이크스트라 등) 중 적합한 것은 한 좌석에서 다른 좌석으로 이동하는 데 항상 1만큼 이동한 것이라고 보기 때문에 BFS입니다.
- ✓ 자료 구조 Queue를 사용하여 BFS를 구현하면 해결할 수 있습니다.
- ✓ 이동 가능한 방향이 상, 하, 좌, 우 4방향이므로, 격자맵을 넘어가거나, 이미 탐색한 좌석을 재방문하지 않도록 조심해야 합니다.



## E. Even하게 익은 스테이크

출제자: 노태운 (@areian13)

태그: #binary\_search, #parametric\_search

예상 난이도: **Silver1**

- ✓ 제출 60번, 정답 5명 (정답률 8.3%)
- ✓ 처음 푼 사람: 김0원 48분

## E. Even하게 익은 스테이크



- ✓ 화력이  $f$  일 때, 스테이크를 모두 굽는 시간  $\text{total\_time}(f) = \sum_{i=1}^n \left\lceil \frac{s_i}{f} \right\rceil$  이고, 우리의 목표는  $\text{total\_time}(f) \leq t$  인 최소 정수  $f$  를 찾는 것입니다.
- ✓ 참고로  $\lceil \cdot \rceil$  은 올림이란 뜻으로  $\lceil 0.0...01 \rceil = 1$  입니다.  
따라서 화력을 무한하게 올리더라도  $\text{total\_time}(\infty) = \sum_{i=1}^n \lceil 0^+ \rceil = n$  이므로,  
 $t < n$  인 경우  $t$  분 이내에 모든 스테이크를 굽는 것은 불가능하여  
이때 -1을 출력하면 됩니다.
- ✓ 추가로 모든  $i$  에 대해  $\left\lceil \frac{s_i}{\max(s)} \right\rceil = 1$  이므로  $\max(s)$  이상의 화력은 의미가 없습니다.



## E. Even하게 익은 스테이크

- ✓ 직관적으로  $f$ 가 증가함에 따라  $\text{total\_time}(f)$ 는 단조 감소하는 그래프임을 알 수 있습니다.

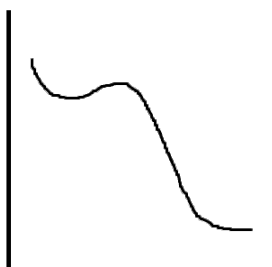
한국말로 풀어쓰면, 화력이 세지면 굽는 시간이 줄어든다는 말입니다.



단조 감소



강한 단조 감소



단조 감소 X



- ✓ 배열 혹은 함수가 단조 감소, 단조 증가한다면 이분 탐색을 사용할 수 있습니다.  
탐색 범위  $\text{start} = 1, \text{end} = \max(s)$  겠네요.  
입력 조건상  $s_i$  는 최대  $10^9$  이니,  $\text{end} = 10^9$  으로 해도 됩니다.  
이렇게 범위를 잡은 후에 이분 탐색을 해주면 됩니다.
- ✓ 시간 복잡도  
이분 탐색으로  $f$  를 결정하는 비용 =  $O(\log 10^9)$   
 $\text{total\_time}(f)$  를 계산하는 비용 =  $O(n)$   
이 둘을 곱한  $O(n \log 10^9)$





## F. 이기적인 별자리

출제자: 노태윤 (@areian13)

태그: #set, #mst, #greedy

예상 난이도: Gold2

- ✓ 제출 10번, 정답 0명 (정답률 0%)
- ✓ 처음 푼 사람: -



- ✓ 일단 좌표가 중복되는 점에 대한 처리부터 해봅시다.
- ✓ 가장 간단한 방법은 모든 점을 표현할 수 있는 2차원 배열을 만드는 겁니다.  
이런! 좌표의 범위는  $-10\,000 \sim 10\,000$ 이니,  $20\,001 \times 20\,001 \approx 4$ 억짜리 배열을 만드는 건 무리일 듯 하네요..
- ✓ 대신 맵을 사용하면 됩니다. 별의 개수는 2,000개이므로, 해시맵이든 트리맵이든 크게 문제 없습니다.
- ✓ 해시맵과 트리맵 모두 편의상 map이라 쓰겠습니다.  
`map<int, map<int, int>> pos`를 선언하여  
`pos[x][y] = max(pos[x][y], b)`로 갱신해주면 됩니다.



- ✓ `map<pair<int, int>, int> pos` 를 쓰는 방법도 있습니다.  
`pos[{x, y}] = max(pos[{x, y}], b)` 이런 식으로 쓰겠습니다.
- ✓ 해시맵을 쓰기 위해선 키 타입에 대해 `hash` 함수와 `equal` 연산자(`==`)가 정의 되어있어야 합니다. 하지만 C++에선 `pair`의 `hash` 함수가 정의 되어있지 않습니다. 따라서 `pair`의 `hash` 함수를 직접 정의해주셔야 합니다.  
언어별 정의 방법은 구글링 해보는 것을 추천합니다
- ✓ 트리맵은 키 타입에 대해 `less` 연산자(`<`)만 정의 되어있으면 됩니다.  
C++는 `pair`에 대해 `less` 연산자가 정의되어 있으므로 별도의 구현 없이 바로 사용할 수 있습니다.



- ✓ 이제 별을 이어봅시다.  
만약 중심별이란 개념이 없었다면, 이 문제는 매우 웰노운한 MST였습니다.  
 $n$ 개의 점에 대해서  $O(n^2)$ 으로 간선들을 만드로 MST를 돌리면 되죠.
- ✓ 허나, 중심별에 의해 무작정 별을 이을 수 없습니다.  
MST를 진행하면서 어떤 별  $u, v$ 를 연결하고자 할 때,  
 $u, v$ 가 각자 속한 별자리(별 집합)에 이미 중심별이 있는가를 판단하고,  
양 쪽 모두 중심별이 있다면 연결해선 안됩니다.
- ✓ 이를 구현하는 방법은 크게 두가지가 있습니다.



## F. 이기적인 별자리

- ✓ 첫번째 방법입니다.  
일단 MST 과정에서  $u, v$  모두 비중심별이라면, 고민할 것 없이 union 해줍니다.
- ✓ 만약  $u$ 는 중심별,  $v$ 는 비중심별이라면,  $u$ 가  $v$ 의 parent가 되도록 분리 집합을 형성합니다.  
즉,  $\text{find}(v) == u$ 가 되도록 구현하면 됩니다.
- ✓ 이후 다른 간선의 별  $u, v$ 를 union하기 전,  
 $\text{find}(u)$ 와  $\text{find}(v)$ 가 모두 중심별이라면 union하지 않고 넘어가면 됩니다.
- ✓ 이렇게 MST를 진행하면  
각각의 분리 집합이 하나의 중심별을 root 노드로 갖는 최소 비용 집합이 됩니다.



- ✓ 첫번째 방법은 직관적이나, 루트 노드가 무엇인지, 비/중심별을 판별하는 등의 구현상의 귀찮음이 많습니다. 두번째 방식은 매우 그리디합니다.
- ✓ 결론적으로, 간선을 생성하는 단계에서 만약 두 별  $u, v$  가 모두 중심별이라면, 간선의 비용을 0으로 설정하면 됩니다.  
이렇게 하고 간선들을 정렬하고 MST를 돌리면, 비용이 0인 중심별들 사이의 간선이 최우선으로 연결됩니다.
- ✓ 이후 이미 어떤 중심별과 연결된 비중심별  $u, v$  가 union을 시도할 때, 개념적으로는 본인들이 속한 별자리가 별개의 별자리일지라도, 이미 중심별들끼리 하나의 집합을 이루고 있으므로 union 되지 않습니다.



- ✓ 아이디어가 중요한 문제였고, 시간 복잡도 자체는 일반적인 MST와 다를 바 없기 때문에

간선을 만드는 비용  $O(n^2) = O(e)$

정렬하는 비용  $O(e \log e)$

MST를 돌리는 비용  $O(e \log n)$

입니다.



## G. 학교 가기 싫어!

출제자: 노태운 (@areian13)

태그: #convex\_hull, #polygon\_area, #rotating\_calipers

예상 난이도: **Platinum2**

- ✓ 제출 4번, 정답 0명 (정답률 0%)
- ✓ 처음 푼 사람: -





- ✓ 문제 이해 자체는 쉬웠지만, 대개 구현이 막혔으리라 생각됩니다.  
다만, 태그에 표기된 알고리즘을 아는 사람 입장에선 응용이 전혀 없는 웰노운입니다.
- ✓ 이 자리에서 볼록 껍질, 다각형의 넓이, 회전하는 캘리퍼스를 설명하는 것은 부적절하므로, 궁금하신 분께선 저 키워드로 직접 공부해보시는 것을 추천드립니다.
- ✓ 각 알고리즘이 어떻게 사용되는지 정도 설명하자면,
  1. 볼록 껍질을 통해  $O(n \log n)$ 으로 모든 점을 포함하는 가장 작은 볼록 다각형을 구합니다.
  2. 신발끈 공식으로 유명한 방식을 통해 다각형의 넓이를  $O(n)$ 으로 구합니다.
  3. 회전하는 캘리퍼스를 이용해 가장 먼 두 점을  $O(n)$ 으로 구할 수 있습니다.